

Vietnamese Traffic Sign Detection and Warning System

Hoang Gia Bao

Abstract

Accurate and real-time traffic sign recognition is a key component of Advanced Driver Assistance Systems (ADAS) and autonomous vehicles, particularly when deployed on resource-constrained edge devices. However, existing solutions often face challenges in Vietnam’s traffic environment due to the scarcity of datasets that realistically reflect local real-world conditions. In this work, I harden the training and evaluation pipeline built on the VR-TSD dataset around three problems uncovered during development. First, I design a two-stage duplicate/leakage screening procedure – combining filename-based provenance matching with perceptual image hashing – to identify images shared between the training set and the pooled validation/test pool, and to guarantee that near-duplicate frames within the pool can never straddle the new validation and test subsets. Second, I identify five traffic-sign categories whose instance counts are too low for the model to learn reliably; keeping them in the taxonomy only hurt training stability and end-user-facing accuracy, so they are pruned, leaving 53 classes, and I apply a bounding-box-aware offline augmentation strategy that selectively multiplies the remaining rare and historically difficult classes. Third, to assess robustness under Vietnam’s adverse driving conditions, I construct two offline, post-hoc stress-test domains from the held-out test set: a procedurally generated haze/fog domain and a physically based windshield-raindrop rendering domain (refraction, Fresnel edge highlights, gravity-fed rivulets), with parameters hand-tuned to stay meaningfully harder than a token weather effect while remaining recoverable in F1. Based on this pipeline, I train a lightweight YOLO11n detector at 896×896 resolution and calibrate per-class confidence thresholds on the validation set. The resulting model reaches a clean-test mAP@0.5 of 96.04% (mAP@0.5:0.95 = 81.97%) at a compact size of 5.3 MB, and clears both predefined robustness targets (mAP@0.5 $\geq$ 85%, Macro F1 $\geq$ 75%) on both stress-test domains, reaching mAP@0.5 = 90.72% / F1 = 75.84% under fog and mAP@0.5 = 90.81% / F1 = 77.07% under rain. Combined with an audio-based warning module and a Streamlit-based web application, and on-device inference at up to 78.74 FPS on a Samsung Galaxy S23, these results confirm the feasibility of the proposed system for real-world ADAS applications tailored to Vietnam’s traffic conditions.

Introduction

Traffic sign detection and recognition is a crucial component of Advanced Driver Assistance Systems (ADAS), supporting traffic safety and enabling various levels of autonomous driving. These systems require both high accuracy and low latency to operate reliably in diverse real-world environments. While vehicle and pedestrian detection often demand high frame rates (≥ 30 FPS), traffic sign recognition – being relatively static – typically requires only about 10 FPS (< 100 ms latency) to ensure timely driver assistance.

Recent research has applied and optimized object detection models such as YOLO, Faster R-CNN, and MobileNet-SSD for traffic sign detection, especially for small and distant signs under challenging conditions. However, most existing studies rely on foreign datasets or limited Vietnamese datasets that lack diversity in sign types, environmental conditions, and traffic scenarios. As a result, current models may not generalize well to the complex characteristics of Vietnamese highways, national roads, and urban traffic.

This study addresses these limitations by proposing a lightweight and efficient traffic sign detection system tailored for Vietnam, with a particular focus on the reliability of the data itself rather than only the model. The main contributions include:

1. **Leakage-aware data preparation.** I design and apply a two-stage duplicate/leakage screen-

ing pipeline – filename-provenance matching combined with perceptual-hash near-duplicate detection – that first screens the training set against the pooled validation/test images, and then, via hash-based union-find clustering, consolidates the validation/test pool itself so that no near-duplicate frame can straddle the two evaluation subsets. This directly addresses the validation/test inconsistency observed in earlier development of this system.

2. **Data-driven class-taxonomy refinement.**

Rather than keeping every category the source dataset defines, I quantify per-class learnability and prune five categories whose instance counts were too small to be learned reliably; retaining them was actively harmful to both training and the driver-facing accuracy of the system, so the final taxonomy is reduced to 53 classes.

3. **Custom offline rare/hard-class augmentation.**

A bounding-box-aware Albumentations pipeline selectively multiplies the remaining under-represented and historically difficult classes toward a target instance count, without touching well-represented classes.

4. **Tuned, physically motivated stress-test domains.**

I build two offline evaluation domains – procedural fog and physically based windshield-

raindrop rendering – and hand-tune their severity parameters against the model’s own F1 response, so that the domains stay clearly harder than a token weather effect while remaining within a recoverable operating range, meeting a target of $\geq 85\%$ mAP@0.5 and $\geq 75\%$ Macro F1 on both.

5. **Edge-optimized, single-model pipeline.** A lightweight YOLO11n model is trained end-to-end on the cleaned and augmented data, with per-class confidence thresholds calibrated on the validation set, and validated on real hardware – reaching 78.74 FPS on a Samsung Galaxy S23 at a 5.3 MB footprint.
6. **End-to-end system.** The trained model is packaged into a Streamlit-based web application (image and video input, no installation required) with a concise audio warning module that conveys the essential meaning of each detected sign to the driver.

Related work

Traffic sign detection is a fundamental problem in computer vision and plays an important role in Advanced Driver Assistance Systems (ADAS) and autonomous vehicles. With the rapid development of deep learning, Convolutional Neural Network (CNN)-based methods have achieved significant improvements in object detection performance. Early two-stage detectors, such as R-CNN and its successors, provided high detection accuracy but suffered from relatively high computational complexity and slow inference speed. To address these limitations, one-stage detectors, particularly the YOLO (You Only Look Once) family, were introduced to achieve a better balance between detection accuracy and inference efficiency.

The YOLO family has undergone continuous architectural improvements over successive generations. Early versions, from YOLOv1 to YOLOv3, established the foundation for real-time object detection and introduced multi-scale feature representations to improve the detection of small objects. Later generations, including YOLOv5, further improved computational efficiency and provided lightweight variants suitable for deployment on resource-constrained devices. YOLOv8 introduced a modern anchor-free detection architecture and further improvements in both accuracy and inference efficiency. More recently, YOLOv10 incorporated architectural optimizations aimed at reducing inference latency and computational overhead, while YOLO11 further improved the trade-off between accuracy, model complexity, and deployment efficiency. Building on this progress, Ultralytics introduced YOLO26, which removes the Non-Maximum Suppression (NMS) post-processing step and the Distribution Focal Loss (DFL) module in favor of a native end-to-end prediction pipeline, together with training refinements aimed at improving small-object detection and inference speed on edge and low-power hardware [15].

Among lightweight YOLO variants, YOLO11n offers a favorable accuracy/size/speed trade-off for edge deployment; at a compact 5.3 MB with 2,592,487 parameters, it is adopted as the target architecture for all of the data-pipeline and robustness work described in the remainder of this report.

Another important limitation of existing traffic sign datasets is their inability to fully represent the diverse and challenging environmental conditions encountered in real-world Vietnamese traffic. Publicly available datasets, such as GTSRB and TT100K, primarily focus on traffic environments from Germany and China, respectively, and may not adequately capture local characteristics or adverse visual conditions in Vietnam. Beyond environmental diversity, a second and less-discussed limitation of Roboflow-style exported datasets is that train/validation/test splits are frequently generated from video-extracted frames without explicit duplicate control, which can silently leak near-identical frames across splits and inflate reported generalization performance. This study treats both problems – environmental diversity and cross-split duplication – as first-class concerns rather than only tuning model architecture.

Dataset and Proposed Model

Base dataset

The original dataset was obtained from the VR-TSD Computer Vision Dataset, which contains 58 traffic sign categories commonly encountered in Vietnam. The dataset consists of 8,078 images with 13,016 manually annotated bounding boxes, released with an original split of 70% training (5,668 images), 16% validation (1,259 images), and 14% test (1,151 images). On average, each image contains one to three traffic signs, while some images contain up to five signs. The original training split is retained unchanged throughout this study – its images are never redistributed into validation or test; only the released validation and test folders are pooled and reorganized, as described next.

Leakage screening and cross-split consolidation

During earlier development of this system, a discrepancy of 9.38 percentage points was observed between validation and test mAP@0.5, which was substantially larger than would be expected from an independent held-out split and raised concerns about duplicate or near-duplicate frames leaking across the original validation/test boundary – a plausible risk given that the source images were extracted from dash-cam video. I therefore built a dedicated screening and consolidation pipeline to resolve this rather than merely re-shuffling the data.

Train vs. pool screening. The pooled validation+test images (2,410 images in total) are treated as the evaluation reference set. Every training image is checked against this pool with two complementary

detectors: (i) a *filename-provenance* check that recovers the shared Roboflow source identifier embedded in each filename, flagging training images that trace back to the same source frame as a pool image; and (ii) a *perceptual-hash* check, using an 8×8 average hash with a Hamming-distance threshold of 4, which additionally catches near-duplicate frames that do not share a filename (e.g., adjacent frames from the same video clip). Images flagged by either detector are treated as leaking evaluation content into training and are excluded from the training set before any model is trained, so that reported validation/test performance reflects genuine generalization rather than memorized near-duplicates.

Stratified re-split with duplicate consolidation. The validation/test pool is then rebuilt independently of the dataset’s original split. I first apply iterative multi-label stratified sampling, targeting 1,410 validation and 1,000 test images so that the per-class label distribution is preserved on both sides. Because this stratified assignment alone cannot guarantee that a near-duplicate frame is not split across validation and test, I additionally run the same perceptual-hash detector *within* the pool and merge matching images into duplicate clusters via a union-find procedure; whenever a cluster straddles validation and test, every image in that cluster is reassigned to whichever side already holds the majority of the cluster. This consolidation step turned out to be far from a minor correction: it reassigned 1,364 of the 2,410 pooled images, indicating substantial frame-level redundancy in the original pool consistent with a video-extraction origin, and produced a final split of 1,927 validation images and 483 test images. A small number of classes (7 of 58) ended up with instances on only one side of this final split; this is reported here as an open limitation (see Conclusion) rather than corrected post hoc, since doing so would have reintroduced the duplication problem the consolidation step is designed to prevent. Unless otherwise noted, all results reported in this study (Tables 2–4 and Figure 2) are computed on this reorganized validation/test split.

Class-taxonomy refinement

Analysis of per-class instance counts and recall across training runs on the original 58-class taxonomy showed that five categories were persistently under-represented to a degree that made them unlearnable in practice: *133-het-cam-vuot*, *421-ket-thuc-khu-dong-dan-cu*, *423a-duong-nguoi-di-bo-sang-ngang-1*, *124b-cam-oto-quay-dau-xe*, and *420-bat-dau-khu-dong-dan-cu*. Rather than letting these five classes sit in the label space contributing near-zero recall while still consuming model capacity and producing unreliable predictions – which, in a driver-facing system, is worse than not attempting the class at all – I removed them from the taxonomy and remapped all remaining class IDs to a contiguous space. This removed 417 training instances, 404 validation instances, and 25 test instances, leaving **53 classes** for all subsequent training and evaluation in this study.

Offline rare- and hard-class augmentation

Even after pruning the five unlearnable classes, the remaining taxonomy still exhibits class imbalance. I flag any class with fewer than 50 training instances as *rare*, and separately designate three classes that had proven historically difficult in prior training runs (*134-het-han-che-toc-do-toi-da*, *103b-cam-oto-re-phai*, *301a-cac-xe-chi-duoc-di-thang*) as *hard*, regardless of their instance count. For every training image containing at least one rare or hard class, I generate additional bounding-box-aware augmented copies using a targeted Albumentations pipeline (shift/scale/rotate, brightness/contrast, hue-saturation-value, and Gaussian blur, with a minimum bounding-box visibility of 0.3), with the number of copies per image scaled inversely to that image’s rarest class count (bounded between 3 and 20 copies) so as to approach a target of 100 instances per class without over-multiplying already-adequate classes. This procedure generated 1,933 synthetic training samples covering 18 rare/hard classes, applied to the training split only.

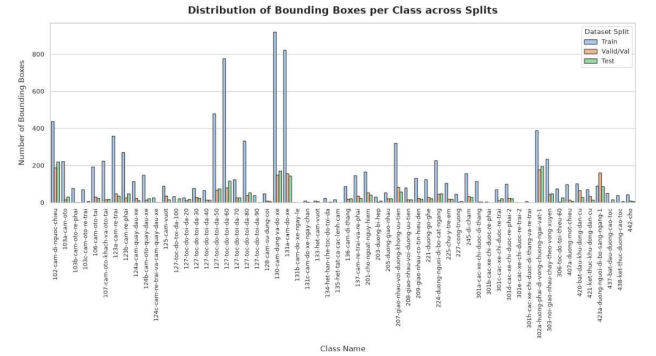


Figure 1: Number of Bounding Boxes of each label in VR-TSD (53-class taxonomy after pruning)

Methodology

The experimental pipeline was organized into ten stages: dataset loading, leakage screening (train vs. pool), stratified re-split with duplicate consolidation, class-taxonomy pruning, offline rare/hard-class augmentation, stress-test domain generation, model training, post-training per-class threshold calibration, multi-domain evaluation, and export/deployment. Experiments were conducted in a Kaggle environment with dual NVIDIA T4 GPU acceleration.

Dataset Loading: The environment was initialized with a fixed random seed (2102) across Python, NumPy, and PyTorch. The original training folder was copied unchanged; the original validation and test folders were pooled for reorganization, as described in the Dataset section above.

Leakage Screening and Re-split: Section “Leakage screening and cross-split consolidation” above details the filename- and hash-based train/pool screening, the iterative multi-label stratified split, and the union-find duplicate-cluster consolidation applied to

the validation/test pool.

Class Pruning and Rare-Class Augmentation: The five persistently unlearnable classes were removed and all label IDs remapped to a contiguous 53-class space; bounding-box-aware Albumentations augmentation was then applied to rare and historically hard classes in the training split only, as described above.

Stress-Test Domain Generation (Evaluation Only): Two offline stress-test domains – Fog and Rain – were generated once from the held-out test split (483 images) and never seen during training. *Fog* uses a procedural haze model: a low-resolution random noise field is Gaussian-blurred and alpha-blended with a uniform haze color, with intensity randomly drawn per image from the range [0.25, 0.50] on top of a base haze thickness of 0.40 and a haze color sampled in [205, 228] (grayscale). This is calibrated to resemble the persistent urban haze and dust commonly observed in major Vietnamese cities rather than dense, opaque fog. *Rain* uses a physically based windshield-raindrop renderer (external `raindrops.py` module) that simulates Snell refraction, Fresnel edge highlights, and gravity-fed rivulets rather than simple overlaid streaks. Its severity parameters (150 drops, up to 20 px radius with a 7% probability of a larger drop up to 40 px, a 30% streak probability, 20 gravity-fed rivulets, and drops extending up to 85% of frame height) were hand-calibrated against the trained model’s own F1 response, so that the domain stays clearly harder than a token rain effect while remaining within the predefined $\geq 75\%$ Macro F1 target.

Model Training: A single YOLO11n model was trained from the COCO-pretrained checkpoint on the cleaned, class-pruned, augmented training set, using only standard Ultralytics online augmentation (mosaic, HSV jitter, translate/scale) – no weather effect is injected during training, so that the Fog and Rain stress-test domains remain a genuinely held-out robustness check rather than a distribution the model was exposed to.

Per-Class Confidence Threshold Calibration: After training, the confidence threshold for each class was calibrated independently on the validation set to maximize its F1 score; classes with fewer than 15 validation instances retained the default threshold of 0.25, since their F1 curves are too noisy to calibrate reliably.

Multi-Domain Evaluation: The calibrated model was evaluated on four domains: clean Validation, clean Test (held-out baseline), and the Fog and Rain stress-test domains, using the per-class thresholds above and an IoU threshold of 0.5 for matching. Performance is reported as mAP@0.5, mAP@0.5:0.95, Precision, Recall, and Macro F1.

Export and Deployment: The trained model was exported to ONNX for portability, and separately to TorchScript/NCNN for on-device benchmarking; it was integrated into a Streamlit-based web application supporting image and video input with visual detection overlays and concise audio warnings.

Training and Deployment Environment

All training experiments were conducted on Kaggle using two NVIDIA T4 GPUs. The final model is based on YOLO11n, trained at 896×896 resolution to better preserve features of small, distant traffic signs. Table 1 summarizes the training configuration; the learning rate is scaled with total batch size (capped at 0.004) and cosine-annealed to a final fraction of 0.01. A deterministic random seed was used throughout to support reproducibility.

Table 1: Hyperparameters for model training (YOLO11n, this study)

Category	Specification	Value
Training environment	GPU	2 × NVIDIA T4
	Image size (imgsz)	896
Training parameters	Maximum epochs	200
	Batch size (total)	64
	Optimizer	AdamW
	Patience (early stopping)	35
	Initial learning rate (lr0)	0.0020
	Final LR fraction (lrf)	0.01
	LR schedule	Cosine
	Warm-up epochs	8
	Box / Cls / DFL loss gain	7.5 / 0.6 / 1.5
	Mosaic (close at epoch)	1.0 (15)
	Translate / Scale	0.05 / 0.5
	HSV (h/s/v)	0.01 / 0.5 / 0.3
	Horizontal / Vertical flip	0.0 / 0.0

For real-world evaluation, the model was deployed via the Ultralytics app on two representative smartphones: the Samsung Galaxy S23 (upper high-end) and the Google Pixel 5 (upper mid-range). The model was exported to TorchScript and subsequently converted into the NCNN framework for mobile deployment; input videos were processed at 1080p resolution. GPU throughput was measured on the same NVIDIA T4 ×2 (Kaggle) environment used for training, reported here alongside the on-device figures for transparency and reproducibility.

Experimental Results

Overall Performance

Table 2 reports the calibrated YOLO11n model’s performance across the four evaluation domains. On the clean test split, the model reaches mAP@0.5 = 96.04% and mAP@0.5:0.95 = 81.97%, with Precision, Recall, and Macro F1 of 87.36%, 89.11%, and 87.29% respectively. Performance on the validation split is close (mAP@0.5 = 95.63%), and notably shows a higher Precision (91.44%) than the test split, which is consistent with the reorganized split rather than an artifact of leakage, given the screening and consolidation procedure described above.

Table 2: YOLO11n performance across evaluation domains (this study, 53 classes)

Domain	mAP@50	mAP@50-95	Precision	Recall	Macro F1
Validation	95.63%	79.54%	91.44%	91.06%	90.79%
Test (Clean)	96.04%	81.97%	87.36%	89.11%	87.29%
Fog (Stress)	90.72%	76.29%	72.75%	84.02%	75.84%
Rain (Stress)	90.81%	74.08%	75.80%	83.23%	77.07%

With a fused footprint of 2,592,487 parameters, 101 layers, and 5.3 MB on disk (PyTorch weights; 10.3 MB after ONNX export), the model remains well suited to memory- and resource-constrained edge deployment while retaining strong clean-domain accuracy.

Table 3: On-device inference throughput of the deployed YOLO11n model. GPU throughput measured on the NVIDIA T4 $\times 2$ (Kaggle) training environment; Google Pixel 5 and Samsung Galaxy S23 measured on-device via the Ultralytics app (NCNN backend).

Platform	ms / image	FPS
NVIDIA T4 $\times 2$ (Kaggle)	5.78	173.01
Google Pixel 5	58.0	17.24
Samsung Galaxy S23	12.7	78.74

Training Behavior

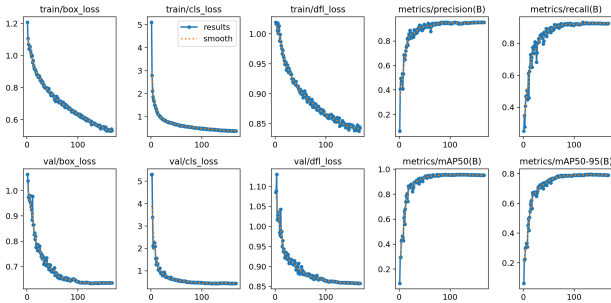


Figure 2: YOLO11n training curves – box/cls/DFL loss and Precision/Recall/mAP per epoch.

Training and validation losses (box, classification, and DFL) decrease steadily across epochs with no sign of divergence, while Precision, Recall, mAP@0.5, and mAP@0.5:0.95 rise quickly in the early epochs and converge toward the domain-level figures reported in Table 2, indicating a stable optimization process without evidence of severe overfitting. The precision-recall curve on the validation split (Figure 4) closes at an overall mAP@0.5 of 0.956, consistent with Table 2, and the F1-confidence curve peaks at approximately 0.93 near a confidence of 0.70 before the per-class threshold calibration is applied – confirming that the calibration step in Section “Per-Class Confidence Threshold Calibration” operates close to each class’s true optimum rather than fighting the underlying curve.

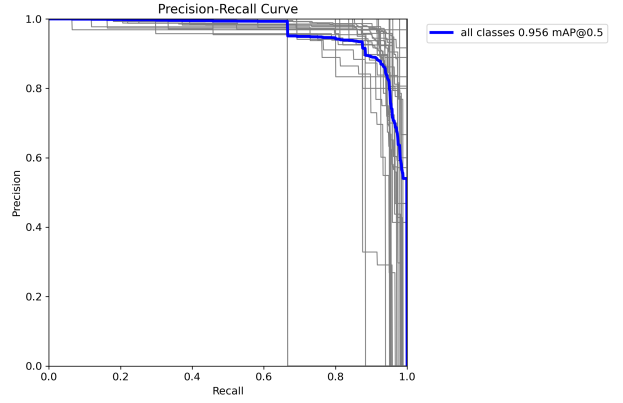


Figure 3: Precision-Recall curve on the validation split (all classes, mAP@0.5 = 0.956).

Robustness Under Fog and Rain

Both predefined robustness targets ($\geq 85\%$ mAP@0.5, $\geq 75\%$ Macro F1) are met on both stress-test domains: Fog reaches mAP@0.5 = 90.72% / F1 = 75.84%, and Rain reaches mAP@0.5 = 90.81% / F1 = 77.07% (Table 2). Both domains show a clear, expected drop relative to the clean test baseline, but the drop is concentrated almost entirely in *Precision* (87.36% clean $\rightarrow$ 72.75% fog / 75.80% rain) rather than *Recall* (89.11% clean $\rightarrow$ 84.02% fog / 83.23% rain), suggesting that most of the degradation comes from additional false positives triggered by haze- and raindrop-induced texture and color artifacts rather than from missed detections. mAP@0.5:0.95 drops more under rain (74.08%) than under fog (76.29%), consistent with the raindrop renderer’s local geometric distortion (refraction and rivulets) degrading precise box localization more than the fog model’s global, purely photometric haze. These results validate the parameter-tuning decisions in the Methodology section: the tuned rain parameters recover enough F1 headroom to clear the 75% target while the domain remains visibly and quantitatively harder than the clean test set.

Conclusion

This study hardens, rather than only re-trains, a lightweight traffic sign recognition system for Vietnam’s road conditions. Two data-quality problems that would otherwise have silently inflated or destabilized results were identified and directly addressed: (i) train/validation/test duplication, resolved via a two-stage filename- and perceptual-hash screening and consolidation pipeline, which reassigned 1,364 of the 2,410 pooled validation/test images to prevent near-duplicate frames from straddling splits; and (ii) severe class under-representation, resolved by pruning five persistently unlearnable categories from the original 58-class VR-TSD taxonomy, leaving 53 classes that the model can actually learn and that the end user can actually rely on.

On top of this cleaned pipeline, a single YOLO11n model – combined with targeted rare/hard-class augmentation and calibrated per-class confidence thresholds – reaches a clean-test mAP@0.5 of 96.04% (mAP@0.5:0.95 = 81.97%) at a 5.3 MB footprint, and clears both predefined robustness targets on two physically motivated, hand-tuned stress-test domains (Fog: mAP@0.5 = 90.72%, F1 = 75.84%; Rain: mAP@0.5 = 90.81%, F1 = 77.07%), with the rain-parameter tuning process in particular illustrating that a stress test’s severity has to be tuned against the model’s own F1 response rather than borrowed as-is from a third-party script. Combined with real-device validation (78.74 FPS on a Samsung Galaxy S23) and a Streamlit-based web application with audio warnings, these results confirm the feasibility of the proposed system for real-world ADAS applications.

Several limitations remain and motivate future work. The consolidation step left 7 classes with instances on only one side of the final validation/test split; enlarging the source pool for those classes, rather than re-shuffling the existing pool, is needed to resolve this without reopening the duplication problem. The system also still struggles with signs that are very distant, heavily occluded, or auxiliary signs carrying textual information. The Fog and Rain domains used in this study are synthetic and, while hand-tuned to be meaningfully difficult, cannot fully substitute for genuine adverse-weather footage. Going forward, I plan to collect additional real-world images for the currently under-represented and pruned sign categories, together with real low-light, foggy, and rainy driving footage, so that both the class taxonomy and the robustness evaluation rely less on synthetic augmentation and better reflect Vietnam’s actual driving conditions; I will also extend on-device testing to platforms such as Raspberry Pi and Jetson Nano.

References

- [1] Yang, Y., Luo, H., Xu, H., & Wu, F. (2015). Towards real-time traffic sign detection and classification. *IEEE Transactions on Intelligent transportation systems*, 17(7), 2022-2031.
- [2] Sarvajcz, K., Ari, L., & Menyhart, J. (2024). Ai on the road: nvidia jetson nano-powered computer vision-based system for real-time pedestrian and priority sign detection. *Applied Sciences*, 14(4), 1440.
- [3] Yang, M., & Han, S. (2025). ETS-YOLO: An Efficient YOLO-based Model for Real-Time Traffic Sign Recognition. *Signal, Image and Video Processing*, 19(8), 650.
- [4] Chaudhuri, A. (2024). Smart traffic management of vehicles using faster R-CNN based deep learning method. *Scientific Reports*, 14(1), 10357.
- [5] Asif, S. M., Anil, T., Tangudu, S., & Keertan, G. S. (2023, May). Traffic Sign Detection Using SSD Mobilenet & Faster RCNN. In *2023 2nd International Conference on Vision Towards Emerging Trends in Communication and Networking Technologies (ViTECoN)* (pp. 1-6).
- [6] Huan, T. P., & Lenh, P. P. C. (2024, November). Vietnamese Traffic Sign Detection for Advanced Driving Assistant: Comparison Between YOLOv8 and Faster RCNN. In *International Conference on Advances in Information and Communication Technology* (pp. 617-628). Cham: Springer Nature Switzerland.
- [7] Vennelakanti, A., Shreya, S., Rajendran, R., Sarkar, D., Muddegowda, D., & Hanagal, P. (2019, January). Traffic sign detection and recognition using a CNN ensemble. In *2019 IEEE international conference on consumer electronics (ICCE)* (pp. 1-4). IEEE.
- [8] Wang, Z., Qu, M., Wang, N., Liu, L., & Su, H. (2025). Lightweight traffic sign detection algorithm based on improved YOLOv5 in snowy environments. *Signal, Image and Video Processing*, 19(5), 1-11.
- [9] Jiang, P., Ergu, D., Liu, F., Cai, Y., & Ma, B. (2022). A Review of Yolo algorithm developments. *Procedia computer science*, 199, 1066-1073.
- [10] Xuan, Y., Zhang, X. Y., Li, C., Wang, H., & Mu, C. (2025). LAM-YOLOv11 for UAV transmission line inspection: Overcoming environmental challenges with enhanced detection efficiency. *Multimedia Systems*, 31(2), 1-13.
- [11] Abdi, N., Parvaresh, F., & Sabahi, M. F. (2024). SeqNet: Sequential networks for one-shot traffic sign recognition with transfer learning. *IEEE Transactions on Intelligent Transportation Systems*.
- [12] VR-TSD dataset: <https://universe.roboflow.com/vietnam-traffic-sign-recognition-benchmark/vr-tsd>
- [13] "Source code. <https://github.com/HoangGiaBao107/Vietnamese-Traffic-Sign-Detection-and-Warning-System>".
- [14] "Link website. <https://vietnamese-traffic-sign-detection-and-warning-system-hgb.streamlit.app/>".
- [15] Ultralytics. YOLO26 – Model documentation. <https://docs.ultralytics.com/models/yolo26/>.